LECTURE TOOLS —
DEEP-DIVE INVESTIGATION SHEET

Only the 5 tools from your wafw00f/Nmap lab session and lecture —
with an escalation ladder for how to push each one further.

LEVEL: Junior / Foundational | SCOPE: Authorized targets only

How to read this sheet

Each tool below has one escalation ladder: **Step 1** is the exact command from your lecture/lab, **Step 2** goes deeper with the same tool's own flags, and **Step 3** shows how to cross-verify or pivot the finding into your next move. Work down each ladder in order — don't jump to Step 3 before you've actually looked at Step 1's output.

```
nslookup → curl -I → wafw00f → Nmap (-sV, NSE scripts) → firewalk
(each tool's findings narrow down what you check next)
```

TOOL 1

nslookup — DNS Reconnaissance

Reveals IP footprint, hosting provider, and load-balancing hints

Step	Command	What it tells you / why you'd run it
Step 1 (baseline)	`nslookup itpro.tv`	Baseline lookup — record every IP address returned. More than one IP is your first load-balancing/CDN signal.
Step 2 (go deeper)	`nslookup -type=NS itpro.tv`	Reveals the authoritative name servers — tells you who hosts DNS (often reveals the cloud/CDN provider directly, e.g. AWS Route53, Cloudflare).
Step 2 (go deeper)	`nslookup -type=TXT itpro.tv`	TXT records often leak which third-party services are in use (SPF entries naming mail providers, domain-verification strings naming cloud platforms).
Step 2 (go deeper)	`nslookup -type=CNAME www.itpro.tv`	If the hostname is a CNAME (alias), the target it points to often directly names the CDN — e.g. ending in .cloudfront.net or .vercel-dns.com.
Step 3 (pivot)	`nslookup <ip-found-above>`	Reverse (PTR) lookup on each discovered IP — the returned hostname frequently reveals the hosting provider even when forward DNS is obscured.

✓ NEXT MOVE

Feed every IP you collect here directly into wafw00f and Nmap by IP address (not just the hostname) — comparing hostname-based vs IP-based results tells you whether the WAF/CDN protection is name-based (virtual host routing) or applied at the network layer.

TOOL 2

curl -I / curl -v — HTTP Header Reconnaissance

Direct evidence: what the server literally tells you, no inference involved

Step	Command	What it tells you / why you'd run it
Step 1 (baseline)	`curl -I https://itpro.tv/`	Grabs response headers only (fast, no body). Look for Server, Via, X-Cache, X-Frame-Options, Content-Security-Policy.

Step	Command	What it tells you / why you'd run it
Step 2 (go deeper)	`curl -v https://itpro.tv/`	Verbose mode — shows the full TLS handshake, request AND response headers, and every Set-Cookie line. Use this to catch WAF-issued cookies (e.g. incap_ses_, AWSALB).
Step 2 (go deeper)	`curl -I -L https://itpro.tv/`	Follows redirects and prints headers at each hop — reveals the full redirect chain and whether different hosts/CDNs appear along the way.
Step 2 (go deeper)	`curl -I -A "Mozilla/5.0" https://itpro.tv/`	Sends a browser-like User-Agent instead of curl's default. Some WAFs/CDNs behave differently (or block outright) based on UA — a mismatch here is itself a finding.
Step 3 (pivot)	`curl -I https://itpro.tv/ (repeat 3-5x)`	Re-run the same request several times and diff the headers/response times. Inconsistency across identical requests is a soft signal you're hitting different backend nodes behind a load balancer.

■ ATTACKER MINDSET

Before running curl -v, predict: given what nslookup already showed you, do you expect to see the same CDN name in the Via header, or something different? Write the prediction down — this is the same “predict then verify” habit from your wafw00f exercise, just applied to a different tool.

TOOL 3

wafw00f — WAF Fingerprinting

Signature-based detection: a hypothesis generator, not a verdict

Step	Command	What it tells you / why you'd run it
Step 1 (baseline)	`wafw00f https://itpro.tv`	Default run — stops at the first matching WAF signature. This is what produced “Vercel WAF” in your lab session.
Step 2 (go deeper)	`wafw00f -a https://itpro.tv`	Finds ALL matching signatures instead of stopping at the first — critical when a site is layered behind more than one provider (e.g. CDN + separate WAF).
Step 2 (go deeper)	`wafw00f -v https://itpro.tv`	Verbose mode — shows you the actual test requests wafw00f sent and which response features triggered the match, so you understand WHY it concluded what it did instead of trusting it blindly.
Step 2 (go deeper)	`wafw00f -r https://itpro.tv`	Disables redirect-following, testing the URL exactly as given — useful when the plain domain redirects (http->https or bare->www) and you want to isolate which hop the WAF sits on.
Step 3 (pivot)	`wafw00f https://<ip-from-nslookup>`	Run wafw00f directly against a raw IP you found earlier. If the result changes or disappears, the WAF is host/SNI-based, not IP-based — an important architectural detail for your notes.

> **✓ NEXT MOVE**
>
> Whatever wafw00f reports, immediately cross-check it against your curl -v cookie/header output from Tool 2. Two independent tools agreeing is what turns a hypothesis into a confirmed finding in your notes.

TOOL 4

Nmap — Service, WAF & Firewall Investigation

One tool, three jobs: banner grabbing, WAF scripts, and firewall evasion flags

Step	Command	What it tells you / why you'd run it
Step 1 (baseline)	`nmap -sV -p 80,443 itpro.tv`	Version/banner grab on the web ports — sometimes leaks load-balancer or server software directly in the service banner.
Step 1 (baseline)	`nmap -p 80,443 --script http-waf-detect itpro.tv`	Nmap's dedicated WAF-detection script — your second independent WAF data point alongside wafw00f.
Step 2 (go deeper)	`nmap -sC -sV -p 80,443 itpro.tv`	Adds Nmap's default safe script set on top of version detection — pulls extra details like HTTP title, methods allowed, and TLS certificate info in one pass.
Step 2 (go deeper)	`nmap --script http-waf-fingerprint itpro.tv`	A more specific NSE script that tries to name the exact WAF product, complementing http-waf-detect's simple yes/no.
Step 3 (firewall probe)	`nmap -f -p 80,443 itpro.tv`	Re-run with packet fragmentation and compare the port states to your Step 1 baseline. A difference tells you the firewall/WAF can (or can't) reassemble fragments before inspecting them.

Step	Command	What it tells you / why you'd run it
Step 3 (firewall probe)	`nmap -sA -p 80,443 itpro.tv`	ACK scan — maps whether the firewall is stateful or just doing simple packet filtering, which shapes how you interpret every other scan's results.

> **■ ATTACKER MINDSET**
>
> Always run the plain baseline scan (Step 1) BEFORE any evasion-flag scan. Without a baseline to compare against, a fragmented or ACK scan's results are meaningless on their own — the finding is always in the difference between the two.

TOOL 5

firewalk — Firewall Rule Mapping

TTL-expiry technique to map exactly which ports survive past the gateway

Step	Command	What it tells you / why you'd run it
Step 1 (baseline)	`firewalk --help`	Review the option list first — confirm which flags your installed version supports before targeting anything.
Step 1 (baseline)	`firewalk -pTCP -d 80 <gateway-ip> <target-ip>`	Sends a TCP probe timed to expire exactly one hop past the gateway. A 'Time Exceeded' reply proves the packet got through the gateway before dying — i.e., port 80 is allowed past it.
Step 2 (go deeper)	`firewalk -pUDP -d 53 <gateway-ip> <target-ip>`	Repeat with UDP against a different port (e.g. DNS/53) — firewall rules are frequently protocol-specific, so a result on TCP tells you nothing about UDP.
Step 2 (go deeper)	`firewalk -S 1-1024 -pTCP <gateway-ip> <target-ip>`	Sweep a port range instead of testing one port at a time — builds a fuller picture of the gateway's allow-list in a single run.
Step 3 (cross-verify)	`nmap --script firewalk <target>`	Nmap's built-in equivalent script. Running both tools against the same target and comparing results is your evidence-based check that the firewall mapping is accurate, not a fluke of one tool's technique.

> **✓ NEXT MOVE**
>
> Write your firewalking results as a simple allow/deny table per port and protocol — that table is exactly what you'll hand off when recommending firewall rule tightening at the end of an engagement.

Putting the 5 tools together

```
nslookup finds IPs + hosting clues
↓
curl -I / -v confirms direct HTTP evidence (headers, cookies)
↓
wafw00f names a likely WAF vendor (signature-based hypothesis)
↓
Nmap cross-checks the WAF finding AND probes firewall behavior
↓
firewalk maps exactly which ports/protocols the gateway allows through
↓
Write findings as: Direct evidence vs. Tool-inferred — confirmed only when 2+ sources agree
```

Source: your live WAFW00F/Nmap lab session against itpro.tv and the IT Pro TV lecture on defense detection. This sheet intentionally covers only the tools that session actually used, each pushed one level deeper than where the lecture stopped.